

LAPORAN TUGAS PRAKTIKUM

STRUKTUR DATA

PEKAN 9

Disusun Oleh:

Khairun Nisa

2511532015

Dosen Pengampu:

DR. WAHYUDI, S.T, M.T

Asisten Pratikum:

Sherly Sukmadira



FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

2026

LAPORAN TUGAS PEKAN 9

1. Source Code Java

```
package pekan9_2511532015;

import java.awt.EventQueue;
import java.awt.Point;
import java.awt.Color;
import java.awt.Font;
import java.awt.Graphics;
import java.awt.Graphics2D;
import java.awt.FontMetrics;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.LinkedList;
import java.util.List;
import java.util.Map;
import java.util.Queue;
import java.util.Stack;

import javax.swing.JFrame;
import javax.swing.JPanel;
import javax.swing.border.EmptyBorder;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.SwingConstants;
import javax.swing.JTextField;
import javax.swing.JTextArea;
import javax.swing.JButton;
import javax.swing.JComboBox;
import javax.swing.DefaultComboBoxModel;

public class petaFTI_2511532015 extends JFrame {

    private static final long serialVersionUID = 1L;
    private JPanel contentPane;

    // Data Graph
    private Map<String, List<String>> adjacencyList_2015 = new
HashMap<>();
    private Map<String, Point> koordinat_2015 = new HashMap<>();
    private PanelGraph_2015 panelGraph_2015;

    // GUI Components
    private JComboBox cbPilAwal_2015;
    private JComboBox cbPilTujuan_2015;
    private JTextArea txtJalur_2015;
    private JTextArea txtNode_2015;
    private JTextField txtJumlah_2015;

    public static void main(String[] args) {
```

```

        EventQueue.invokeLater(new Runnable() {
            public void run() {
                try {
                    petaFTI_2511532015 frame = new
petaFTI_2511532015();
                    frame.setVisible(true);
                } catch (Exception e) {
                    e.printStackTrace();
                }
            }
        });
    }

    public petaFTI_2511532015() {
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setBounds(100, 100, 686, 709);
        contentPane = new JPanel();
        contentPane.setBackground(new Color(255, 255, 255));
        contentPane.setBorder(new EmptyBorder(5, 5, 5, 5));
        setContentPane(contentPane);
        contentPane.setLayout(null);

        //lokasi awal
        JLabel lokasiAwal_2015 = new JLabel("Lokasi Awal :");
        lokasiAwal_2015.setFont(new Font("SansSerif", Font.BOLD,
12));
        lokasiAwal_2015.setBounds(20, 50, 100, 22);
        contentPane.add(lokasiAwal_2015);

        cbPilAwal_2015 = new JComboBox();
        cbPilAwal_2015.setModel(new DefaultComboBoxModel(new
String[] { "Lobi", "Parkiran Motor", "Parkiran Mobil", "Ruang
Dosen", "Mushalla", "Toilet", "Lab DS", "Lab AI", "Kesiswaan",
"PKM"
}));
        cbPilAwal_2015.setBounds(120, 50, 130, 22);
        contentPane.add(cbPilAwal_2015);

        //bfs
        JButton btnBfs_2015 = new JButton("[ BFS ]");
        btnBfs_2015.setForeground(new Color(0, 0, 0));
        btnBfs_2015.setBackground(new Color(0, 102, 204));
        btnBfs_2015.setBounds(366, 50, 80, 25);
        contentPane.add(btnBfs_2015);

        // LOKASI TUJUAN
        JLabel lokasiTujuan_2015 = new JLabel("Lokasi Tujuan :");
        lokasiTujuan_2015.setFont(new Font("SansSerif", Font.BOLD,
12));
        lokasiTujuan_2015.setBounds(20, 85, 100, 22);
        contentPane.add(lokasiTujuan_2015);

        cbPilTujuan_2015 = new JComboBox();
        cbPilTujuan_2015.setModel(new DefaultComboBoxModel(new
String[] { "Lobi", "Parkiran Motor", "Parkiran Mobil", "Ruang
Dosen", "Mushalla", "Toilet", "Lab DS", "Lab AI", "Kesiswaan",
"PKM"
}));

```

```

    }));
    cbPilTujuan_2015.setBounds(120, 85, 130, 22);
    contentPane.add(cbPilTujuan_2015);

    //dfs
    JButton btnDfs_2015 = new JButton("[ DFS ]");
    btnDfs_2015.setForeground(new Color(0, 0, 0));
    btnDfs_2015.setBackground(new Color(255, 0, 255));
    btnDfs_2015.setBounds(456, 50, 80, 25);
    contentPane.add(btnDfs_2015);

    //reset
    JButton btnReset_2015 = new JButton("[ RESET ]");
    btnReset_2015.setForeground(new Color(0, 0, 0));
    btnReset_2015.setBackground(new Color(204, 0, 0));
    btnReset_2015.setBounds(546, 50, 90, 25);
    contentPane.add(btnReset_2015);

    //panel graph
    panelGraph_2015 = new PanelGraph_2015();
    panelGraph_2015.setBounds(10, 125, 577, 356);
    contentPane.add(panelGraph_2015);

    //panel hasil
    JPanel hasil_2015 = new JPanel();
    hasil_2015.setBounds(10, 516, 662, 145);
    hasil_2015.setLayout(null);
    hasil_2015.setBackground(new Color(192, 192, 192));
    contentPane.add(hasil_2015);

    //area jalur
    JLabel lblJalur_2015 = new JLabel("Jalur
:");
    lblJalur_2015.setFont(new Font("SansSerif", Font.BOLD,
12));
    lblJalur_2015.setBounds(10, 10, 120, 20);
    hasil_2015.add(lblJalur_2015);

    txtJalur_2015 = new JTextArea();
    txtJalur_2015.setEditable(false);
    txtJalur_2015.setLineWrap(true);
    txtJalur_2015.setWrapStyleWord(true);
    txtJalur_2015.setBounds(120, 10, 511, 35);
    hasil_2015.add(txtJalur_2015);

    //Area Node Dikunjungi
    JLabel lblNode_2015 = new JLabel("Node Dikunjungi :");
    lblNode_2015.setFont(new Font("SansSerif", Font.BOLD, 12));
    lblNode_2015.setBounds(10, 55, 120, 20);
    hasil_2015.add(lblNode_2015);

    txtNode_2015 = new JTextArea();
    txtNode_2015.setEditable(false);
    txtNode_2015.setLineWrap(true);
    txtNode_2015.setWrapStyleWord(true);
    txtNode_2015.setBounds(120, 55, 511, 45);
    hasil_2015.add(txtNode_2015);

```

```

//Area Jumlah Node
JLabel lblJumlah_2015 = new JLabel("Jumlah Node      :");
lblJumlah_2015.setFont(new Font("SansSerif", Font.BOLD,
12));
lblJumlah_2015.setBounds(10, 110, 120, 20);
hasil_2015.add(lblJumlah_2015);

txtJumlah_2015 = new JTextField();
txtJumlah_2015.setEditable(false);
txtJumlah_2015.setBounds(120, 110, 50, 25);
hasil_2015.add(txtJumlah_2015);

JPanel panel = new JPanel();
panel.setBackground(new Color(0, 64, 128));
panel.setBounds(0, 10, 672, 29);
contentPane.add(panel);

//judul
JLabel txtJudul = new JLabel("PENCARIAN JALUR MENGGUNAKAN
BFS DAN DFS");
panel.add(txtJudul);
txtJudul.setBackground(new Color(0, 64, 128));
txtJudul.setFont(new Font("SansSerif", Font.BOLD, 14));
txtJudul.setHorizontalAlignment(SwingConstants.CENTER);

//Inisialisasi Graph
inisialisasiGraph_2015();

//Action Listener BFS
btnBfs_2015.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        String awal = (String)
cbPilAwal_2015.getSelectedItem();
        String tujuan = (String)
cbPilTujuan_2015.getSelectedItem();
        if (awal.equals(tujuan)) {
            JOptionPane.showMessageDialog(null, "Lokasi
awal dan tujuan tidak boleh sama!");
            return;
        }
        List<String> hasil = bfs_2015(awal, tujuan);
        tampilkanHasil_2015(hasil);
    }
});

//Action Listener DFS
btnDfs_2015.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        String awal = (String)
cbPilAwal_2015.getSelectedItem();
        String tujuan = (String)
cbPilTujuan_2015.getSelectedItem();
        if (awal.equals(tujuan)) {
            JOptionPane.showMessageDialog(null, "Lokasi
awal dan tujuan tidak boleh sama!");
            return;
        }
    }
});

```

```

    }
    List<String> hasil = dfs_2015(awal, tujuan);
    tampilkanHasil_2015(hasil);
    }
});

//Action Listener Reset
btnReset_2015.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        panelGraph_2015.resetColors_2015();
        txtJalur_2015.setText("");
        txtNode_2015.setText("");
        txtJumlah_2015.setText("");
    }
});
}

private void tambahEdge_2015(String node1, String node2) {
    adjacencyList_2015.computeIfAbsent(node1, k -> new
ArrayList<>()).add(node2);
    adjacencyList_2015.computeIfAbsent(node2, k -> new
ArrayList<>()).add(node1);
}

private void inisialisasiGraph_2015() {
    //Koordinat 10 Node
    koordinat_2015.put("Lobi", new Point(230, 230));
    koordinat_2015.put("PKM", new Point(280, 35));
    koordinat_2015.put("Kesiswaan", new Point(220, 120));
    koordinat_2015.put("Ruang Dosen", new Point(60, 180));
    koordinat_2015.put("Parkiran Motor", new Point(60, 260));
    koordinat_2015.put("Parkiran Mobil", new Point(300, 325));
    koordinat_2015.put("Mushalla", new Point(400, 150));
    koordinat_2015.put("Toilet", new Point(400, 80));
    koordinat_2015.put("Lab DS", new Point(520, 190));
    koordinat_2015.put("Lab AI", new Point(520, 260));

    //15 Edge
    tambahEdge_2015("PKM", "Kesiswaan");
    tambahEdge_2015("PKM", "Ruang Dosen");
    tambahEdge_2015("Kesiswaan", "Lobi");
    tambahEdge_2015("Ruang Dosen", "Lobi");
    tambahEdge_2015("Ruang Dosen", "Parkiran Motor");
    tambahEdge_2015("Lobi", "Parkiran Motor");
    tambahEdge_2015("Lobi", "Parkiran Mobil");
    tambahEdge_2015("Lobi", "Mushalla");
    tambahEdge_2015("Lab DS", "Toilet");
    tambahEdge_2015("Mushalla", "Lab DS");
    tambahEdge_2015("Lab DS", "Lab AI");
    tambahEdge_2015("Parkiran Mobil", "Lab DS");
    tambahEdge_2015("Parkiran Motor", "Parkiran Mobil");
    tambahEdge_2015("Mushalla", "Toilet");
    tambahEdge_2015("Mushalla", "Parkiran Mobil");

    panelGraph_2015.setDataGraph_2015(koordinat_2015,
adjacencyList_2015);
}

```

```

private List<String> bfs_2015(String awal, String tujuan) {
    Map<String, String> parent_2015 = new HashMap<>();
    List<String> visited_2015 = new ArrayList<>();
    Queue<String> queue_2015 = new LinkedList<>();

    queue_2015.add(awal);
    parent_2015.put(awal, null);

    while (!queue_2015.isEmpty()) {
        String current = queue_2015.poll();
        visited_2015.add(current);

        if (current.equals(tujuan)) {
            panelGraph_2015.setVisitedNodes_2015(visited_2015);
            return reconstructPath_2015(parent_2015, tujuan);
        }

        if (adjacencyList_2015.containsKey(current)) {
            for (String neighbor :
adjacencyList_2015.get(current)) {
                if (!parent_2015.containsKey(neighbor)) {
                    parent_2015.put(neighbor, current);
                    queue_2015.add(neighbor);
                }
            }
        }
    }
    panelGraph_2015.setVisitedNodes_2015(visited_2015);
    return new ArrayList<>();
}

private List<String> dfs_2015(String awal, String tujuan) {
    Map<String, String> parent_2015 = new HashMap<>();
    List<String> visited_2015 = new ArrayList<>();
    Stack<String> stack_2015 = new Stack<>();

    stack_2015.push(awal);
    parent_2015.put(awal, null);

    while (!stack_2015.isEmpty()) {
        String current = stack_2015.pop();

        if (!visited_2015.contains(current)) {
            visited_2015.add(current);

            if (current.equals(tujuan)) {
                panelGraph_2015.setVisitedNodes_2015(visited_2015);
                return reconstructPath_2015(parent_2015,
tujuan);
            }

            if (adjacencyList_2015.containsKey(current)) {
                for (String neighbor :
adjacencyList_2015.get(current)) {
                    if (!parent_2015.containsKey(neighbor)) {
                        parent_2015.put(neighbor, current);
                        stack_2015.push(neighbor);
                    }
                }
            }
        }
    }
}

```

```

    }
    }
    }
    }
    }
    panelGraph_2015.setVisitedNodes_2015(visited_2015);
    return new ArrayList<>();
}

private List<String> reconstructPath_2015(Map<String, String>
parent_2015, String tujuan) {
    List<String> path_2015 = new ArrayList<>();
    String current = tujuan;
    while (current != null) {
        path_2015.add(0, current);
        current = parent_2015.get(current);
    }
    panelGraph_2015.setPathNodes_2015(path_2015);
    return path_2015;
}

//menampilkan hasil
private void tampilkanHasil_2015(List<String> jalur) {
    if (jalur.isEmpty()) {
        txtJalur_2015.setText("Jalur tidak ditemukan!");
        txtNode_2015.setText("-");
        txtJumlah_2015.setText("0");
        return;
    }

    StringBuilder jalurText = new StringBuilder();
    for (int i = 0; i < jalur.size(); i++) {
        jalurText.append(jalur.get(i));
        if (i < jalur.size() - 1) jalurText.append(" -> ");
    }
    txtJalur_2015.setText(jalurText.toString());

    List<String> visited =
panelGraph_2015.getVisitedNodes_2015();
    StringBuilder visitedText = new StringBuilder();
    for (int i = 0; i < visited.size(); i++) {
        visitedText.append(visited.get(i));
        if (i < visited.size() - 1) visitedText.append(", ");
    }
    txtNode_2015.setText(visitedText.toString());
    txtJumlah_2015.setText(String.valueOf(visited.size()));
}

class PanelGraph_2015 extends JPanel {

    private Map<String, Point> koordinat_2015;
    private Map<String, List<String>> adjacencyList_2015;
    private List<String> visitedNodes_2015 = new ArrayList<>();
    private List<String> pathNodes_2015 = new ArrayList<>();

    public PanelGraph_2015() {
        super();
        this.setBackground(Color.WHITE);
    }
}

```



```

    }

    public void setDataGraph_2015(Map<String, Point> koordinat,
Map<String, List<String>> graph) {
        this.koordinat_2015 = koordinat;
        this.adjacencyList_2015 = graph;
        this.repaint();
    }

    public void setVisitedNodes_2015(List<String> visited) {
        this.visitedNodes_2015 = new ArrayList<>(visited);
        this.repaint();
    }

    public void setPathNodes_2015(List<String> path) {
        this.pathNodes_2015 = new ArrayList<>(path);
        this.repaint();
    }

    public void resetColors_2015() {
        this.visitedNodes_2015.clear();
        this.pathNodes_2015.clear();
        this.repaint();
    }

    public List<String> getVisitedNodes_2015() {
        return this.visitedNodes_2015;
    }

    public List<String> getPathNodes_2015() {
        return this.pathNodes_2015;
    }

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        if (koordinat_2015 == null || adjacencyList_2015 ==
null) return;

        Graphics2D g2d = (Graphics2D) g;

        g2d.setRenderingHint(java.awt.RenderingHints.KEY_ANTIALIASING,
java.awt.RenderingHints.VALUE_ANTIALIAS_ON);

        //Gambar Garis (Edge)
        g2d.setColor(Color.LIGHT_GRAY);
        g2d.setStroke(new java.awt.BasicStroke(2));

        for (String node : adjacencyList_2015.keySet()) {
            Point p1 = koordinat_2015.get(node);
            if (p1 == null) continue;

            for (String neighbor :
adjacencyList_2015.get(node)) {
                Point p2 = koordinat_2015.get(neighbor);
                if (p2 == null) continue;
            }
        }
    }

```

```

        g2d.drawLine(p1.x, p1.y, p2.x, p2.y);
    }
}

//Gambar Lingkaran (Node)
int lebarNode = 75;    // Lebih lebar
int tinggiNode = 60;

for (String node : koordinat_2015.keySet()) {
    Point p = koordinat_2015.get(node);
    if (p == null) continue;

    int x = p.x - (lebarNode / 2);
    int y = p.y - (tinggiNode / 2);

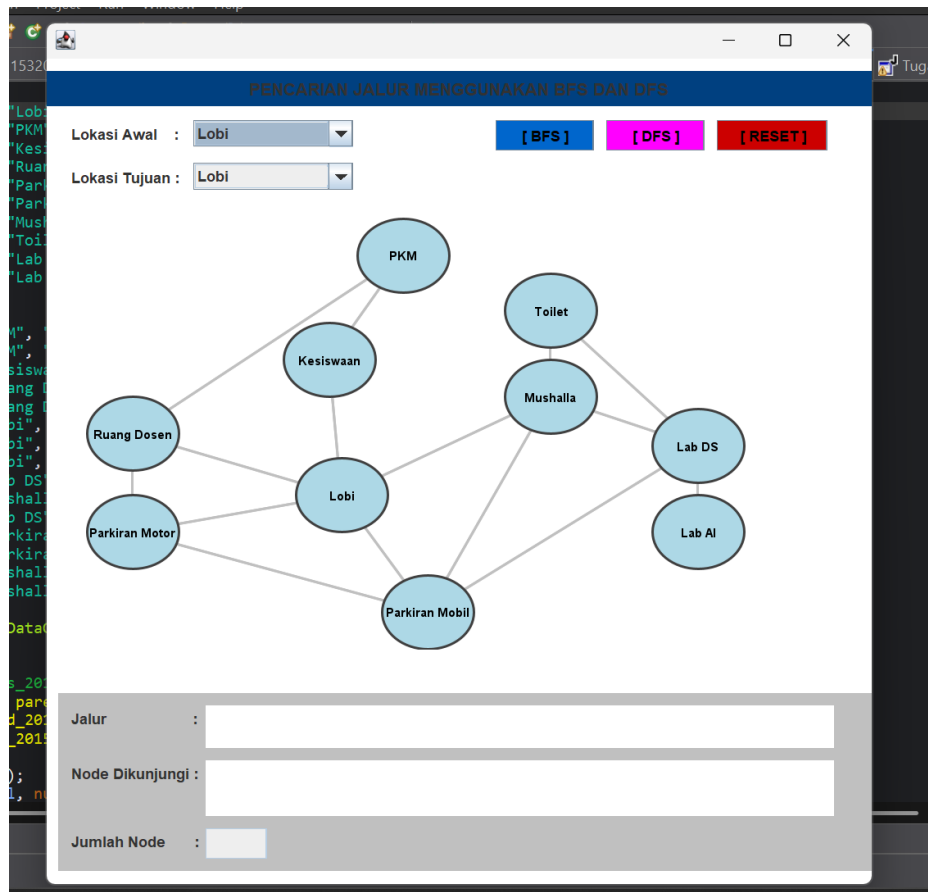
    //Pewarnaan Node
    if (pathNodes_2015.contains(node)) {
        g2d.setColor(new Color(50, 205, 50)); // Hijau
- Jalur
    } else if (visitedNodes_2015.contains(node)) {
        g2d.setColor(new Color(255, 165, 0)); // Oranye
- Dikunjungi
    } else {
        g2d.setColor(new Color(173, 216, 230)); // Biru
- Default
    }

    g2d.fillOval(x, y, lebarNode, tinggiNode);
    g2d.setColor(Color.DARK_GRAY);
    g2d.drawOval(x, y, lebarNode, tinggiNode);

    //Teks Nama Lokasi
    g2d.setColor(Color.BLACK);
    g2d.setFont(new Font("SansSerif", Font.BOLD, 10));
    FontMetrics fm = g2d.getFontMetrics();
    int textWidth = fm.stringWidth(node);
    g2d.drawString(node, p.x - (textWidth / 2), p.y +
4);
    }
}
}
}

```

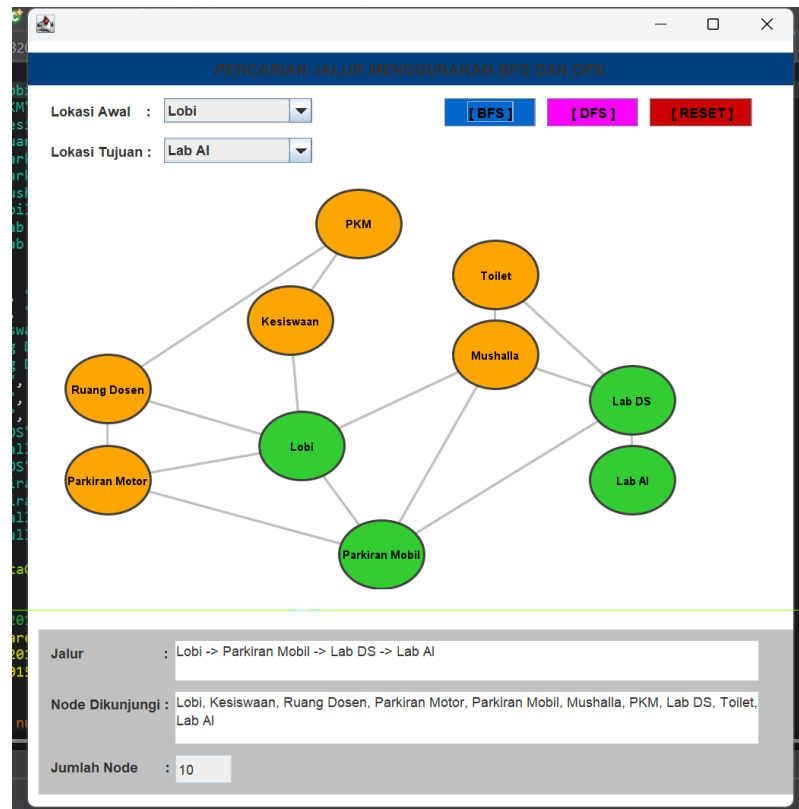
2. Screenshot program saat dijalankan



Gambar 1.1: Hasil program ketika dijalankan pertama kali

3. Penjelasan

- Deskripsi Peta: peta yang dibuat merepresentasikan Gedung Fakultas Teknologi Informasi, Universitas Andalas. Peta ini menggambarkan denah Lokasi-lokasi yang sering saya lalui di FTI beserta jalur penghubung antar Lokasi.
- Jumlah Node dan Edge:
Total Node (Simpul): 10 node
Total Edge (Sisi): 15 edge
- Hasil BFS (Lobi ke Lab AI)



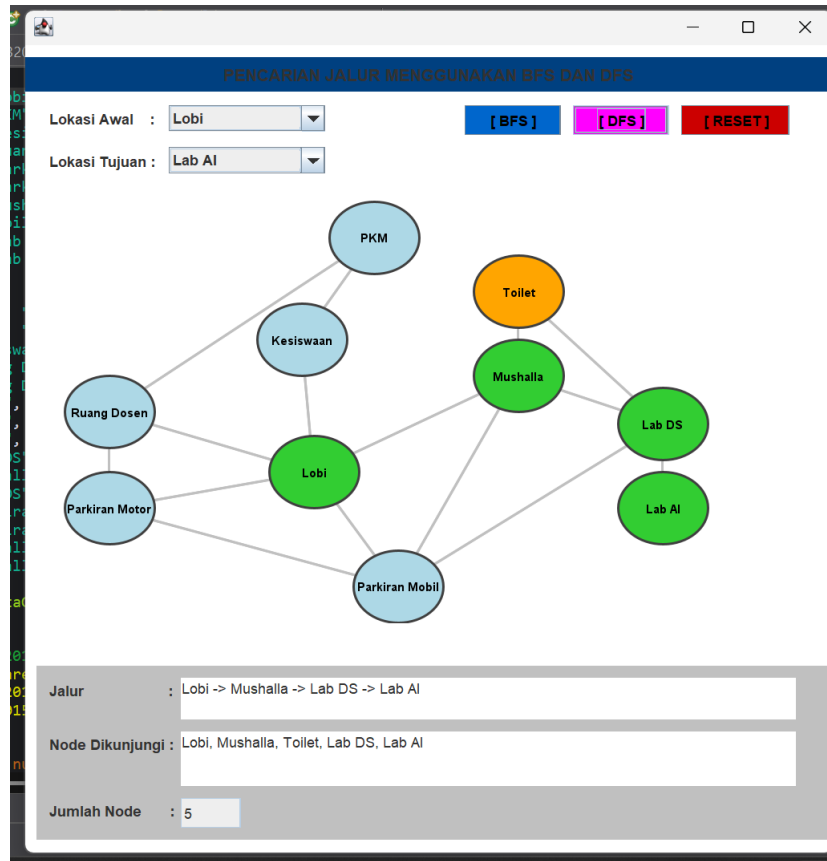
Gambar 3.1 Hasil pemcarian Metode BFS

BFS menjamin menemukan jalur terpendek (dalam jumlah edge paling sedikit). Dari Lobi ke Lab AI, jalur terpendek adalah melalui 3 langkah (3 edge):

1. Lobi → Parkiran Mobil
2. Parkiran Mobil → Lab DS
3. Lab DS → Lab AI

BFS mengeksplorasi node secara melebar (level by level), sehingga semua tetangga Lobi dikunjungi terlebih dahulu sebelum melanjutkan ke level berikutnya.

- Hasil DFS (Lobi ke Lab AI)



Gambar 3.2: Hasil pencarian method DFS

Pada pengujian algoritma DFS dengan titik awal Lobi dan titik tujuan Lab AI, program berhasil menemukan jalur: Lobi → Mushalla → Lab DS → Lab AI.

Algoritma DFS bekerja dengan cara menelusuri graph secara mendalam (depth-first) menggunakan struktur data Stack. Program mengeksplorasi cabang Mushalla terlebih dahulu hingga menemukan tujuan. Total simpul (node) yang dieksplorasi oleh algoritma DFS pada pencarian ini sebanyak 5 node (Lobi, Mushalla, Toilet, Lab DS, Lab AI). DFS tidak selalu menjamin jalur terpendek, namun pada kasus ini DFS menemukan jalur yang cukup efisien dengan mengeksplorasi node yang lebih sedikit dibandingkan BFS.

- Kesimpulan perbandingan BFS dan DFS:
Berdasarkan pengujian yang telah dilakukan pada program pencarian jalur di Gedung Pancasagoro FTI, berikut adalah perbandingan langsung antara algoritma Breadth First Search (BFS) dan Depth First Search (DFS):

1. Strategi Pencarian: Jika menggunakan BFS, algoritma akan menelusuri graf secara melebar (level by level), di mana semua simpul tetangga terdekat dikunjungi terlebih dahulu sebelum berpindah ke level berikutnya. Sedangkan jika menggunakan DFS, algoritma akan menelusuri graf secara mendalam (depth-first), di mana satu cabang ditelusuri sampai ke ujung paling dalam sebelum kembali (backtrack) ke cabang lainnya.
2. Struktur Data: Jika menggunakan BFS, program memanfaatkan struktur data Queue (Antrian) dengan prinsip First In First Out (FIFO). Sedangkan jika menggunakan DFS, program memanfaatkan struktur data Stack (Tumpukan) dengan prinsip Last In First Out (LIFO).

Berdasarkan hasil pengujian pada program, ketika melakukan pencarian jalur dari Lobi ke Lab AI, algoritma BFS mengeksplorasi sebanyak 9 simpul untuk menemukan jalur Lobi → Parkiran Mobil → Lab DS → Lab AI. Di sisi lain, algoritma DFS hanya mengeksplorasi sebanyak 5 simpul untuk menemukan jalur Lobi → Mushalla → Lab DS → Lab AI. Dari hasil tersebut terlihat bahwa pada kasus ini DFS lebih efisien dalam hal jumlah simpul yang dieksplorasi. Namun, hal ini sangat bergantung pada urutan penumpukan stack dan struktur graf. Jika struktur graf berubah, DFS belum tentu menghasilkan jalur seefisien itu, sedangkan BFS tetap akan selalu konsisten menghasilkan jalur terpendek.